

ML 2025 Project



Project Type: **A** (developed in Python)

Master Degree in Data Science and Business Informatics

Candidates – 🐎 iCavalli Team

Albani Lorenzo	<u>l.albani2@studenti.unipi.it</u> - [690625]
Maitino Tommaso	<u>t.maitino@studenti.unipi.it</u> - [582479]
Roselli Riccardo	<u>r.roselli1@studenti.unipi.it</u> - [614720]

Academic Year 2025 / 2026 - Date 23/01/2026

Objectives

Aim

Design and Analyze a fully custom **Neural Network framework** from scratch (NumPy-only), and evaluate its performance both on the *MONK benchmark classification* and *CUP benchmark regression* problems.

Model & Algorithms Overview

- Feed-forward Multilayer Perceptron (**MLP**)
- Trained with **Backpropagation**, **SGD / Adam** optimization, **Activation** functions, **Dropout**, and **Regularizers**
- **Early stopping** driven by validation loss
- Rigorous hyperparameter selection (**coarse** → **refined nested grid search**)
- Robust final evaluation via **multi-seed training** and **ensemble voting**

Our Contributions - Code & Implementation

```
nn/  
  core.py  
  model.py  
  layers.py  
  activations.py  
  dropout.py  
  losses.py  
  optim.py  
  regularizers.py  
  metrics.py  
  callbacks.py  
  
training/  
  config.py  
  dataloader.py  
  history.py  
  trainer.py  
  holdout_cv.py  
  kfold_cv.py  
  model_factory.py  
  refine_grid.py  
  gridsearch.py  
  model_selection.py  
  
data/  
  data_handler/  
    data_loader.py  
  experiments/  
    cup/  
    monk/  
      common.py  
      configs/  
        monk1.yaml  
      results/  
        monk1/  
          best_config.json  
  testing/  
    ensemble_utils.py  
  notebooks/  
    run_experiments.ipynb  
    run_test.ipynb
```

Custom **Neural Network** framework

with modular software design:

- *model definition*
- *training engine*
- *model selection*
- *data and data handling*
- *experiments with grids and results*
- *final testing and evaluation*

Our Contributions - Model & Training

 nn/
core.py
model.py
layers.py
activations.py
dropout.py
losses.py
optim.py
regularizers.py
metrics.py
callbacks.py

Model: Feed-forward Multilayer Perceptron with Dense hidden layers

Activations: Sigmoid, Tanh, ReLU, Softmax for multiclass, Identity for regression

Training algorithms: Backpropagation with SGD, SGD + momentum, or Adam

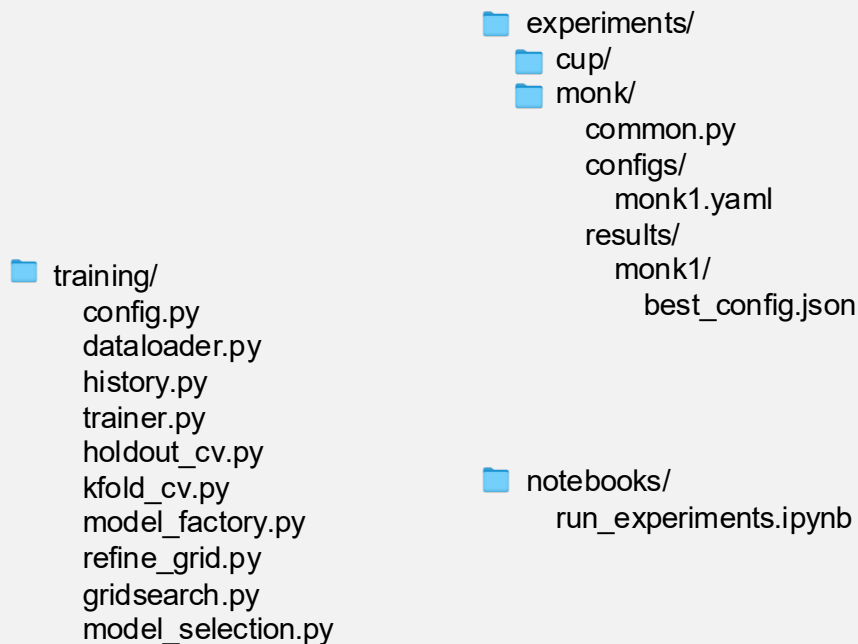
Learning mode: Mini-batch gradient descent

Initializations: Xavier, He, Zeros

Regularizations: L1 and L2 weight decay, Dropout

Stopping conditions: Early Stopping with patience based on validation loss, and best-weight restoration

Our Contributions - Validation Schema (Model Selection & Evaluation)



Our **Multi-Step GridSearch** consisted in **Two-phase validation strategies**:

- **Coarse phase**

- **Holdout validation with 80% Training / 20% Validation split**

- **Refined phase**

- **K-fold Cross-Validation (K = 5)**

After **Model Selection** we evaluated the top final "best model"

- **Test set (blind) used only once**

Our Contributions - Validation Schema (Hyperparameter Exploration)

The initial **Coarse GridSearch** explored a wide and sufficiently discriminative hyperparameter ranges to identify promising regions of the search space from the `yaml` file:

- **Network Depth:** 1 / 2 / 3 hidden layers
- **Hidden Units:** small-to-medium sizes [2 ... 64]
- **Activations:** Tanh, ReLU, Sigmoid
- **Optimizers:** SGD, SGD + momentum, Adam
- **Learning Rate:** 0.3 .. 0.001
- **Reg:** L1, L2 [0 .. 10^{-6}], Dropout [0 .. 20%]
- **Batch Size:** 8, 16, 32
- **Epochs:** 800, 1000

The **Refined GridSearch** was automatically built from **Top-K (K=10) configurations** selected in the coarse phase (based on validation loss) to reduce search space dimensionality on the highest performing regions.

We retained all the categorical hyperparameters (hidden units, activation, optimizers), while we shrunk the continuous hyperparameter regions (learning rate, l2 regularization, dropout, momentum):

- Extract **min / max** values observed in Top-K
- Generate **5-steps linearly spaced values** in the interval

Our Contributions - Final Test Phase (Full Retraining and Testing)

The criterion for the **final model selection** was based on the **best-performing hyperparameter configuration** using the **K-fold CV** in the **refined GridSearch**:

- Selection Metric**: lowest mean validation loss across folds
- Final model hyperparameters**: stored in `best_config.json`, (immutable for final evaluation) which includes the full network architecture (layers, activation, optimizer ...)
- No further tuning** is performed after this point

The final selected model is **retrained from scratch** and then **tested finally** once

- ReTraining**: uses the **entire training set**
- Multi-seed**: **9 independent runs** with different seeds, leads to different weight initialization and mini-batch ordering between runs
- Ensemble construction**: all trained models outputs **class probabilities**, and the final prediction is obtained via **majority voting** (based on BCE on mean probabilities) across the different models
- Best Training Epoch**: for **each seed** we run **5-fold CV on training data** and extract **best epoch per fold** (based on validation loss), then we average **best epochs across folds** per model, and the final epoch for all the models is the **mean of best epochs across seeds**

Our Contributions - Computational Costs

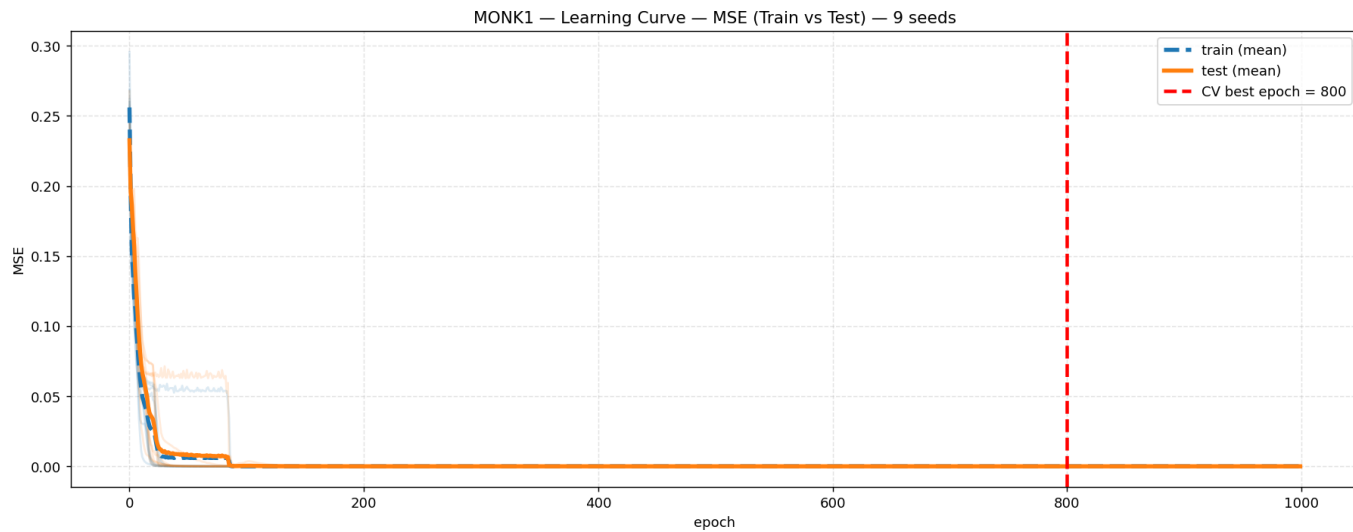
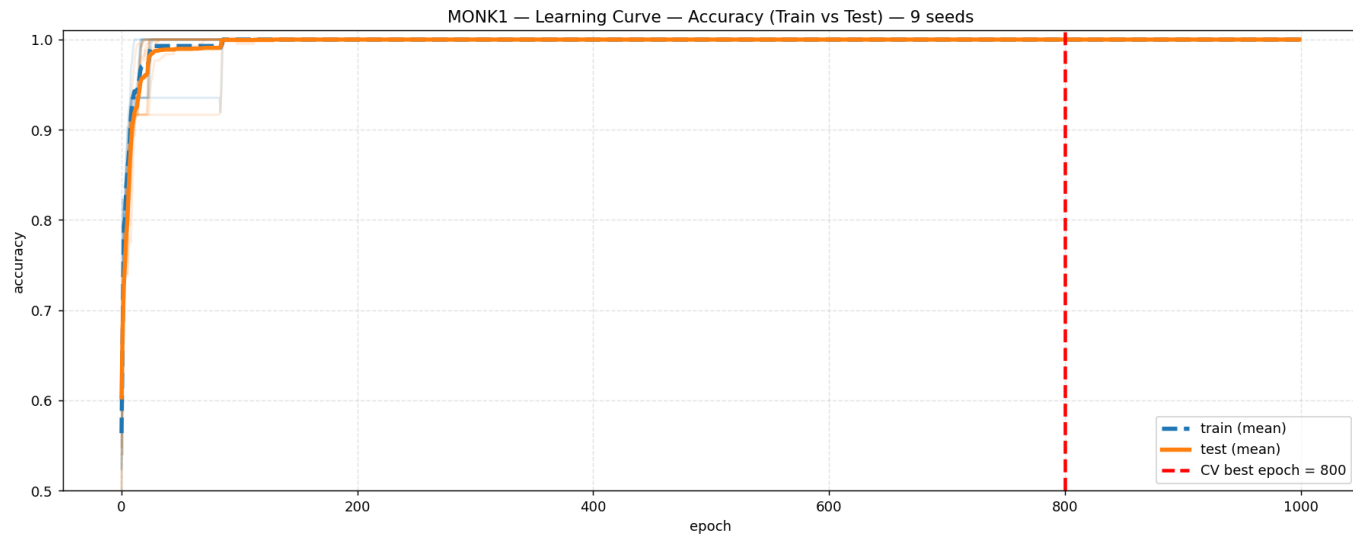


The Training was performed on **CPU** using a Parallel **multi-process execution** (*njobs=-1*) on the GridSearch

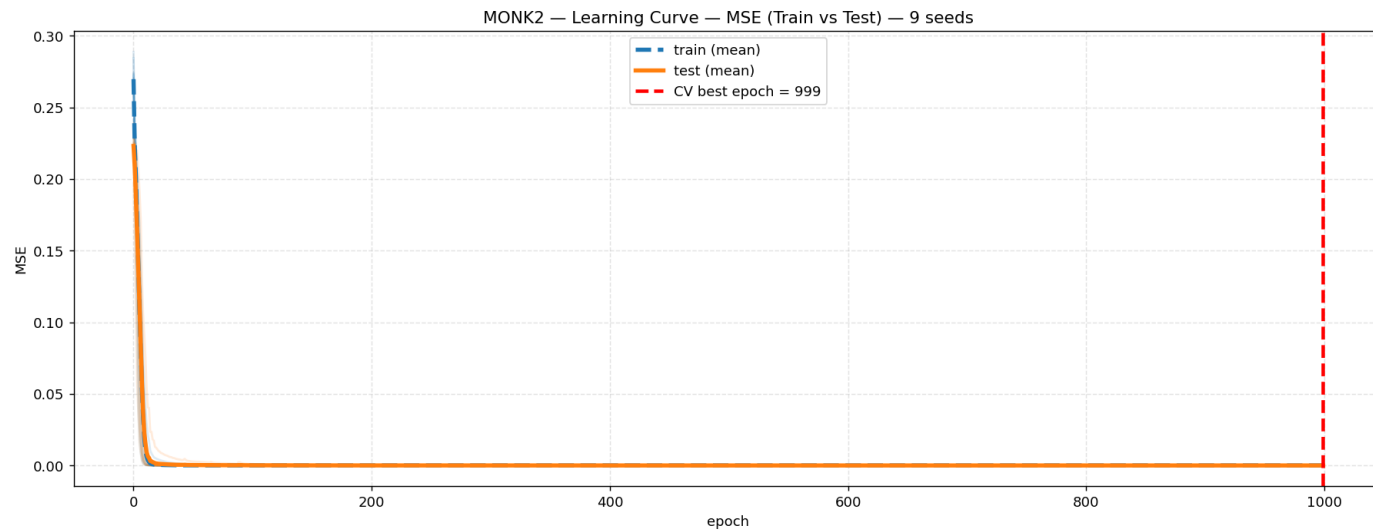
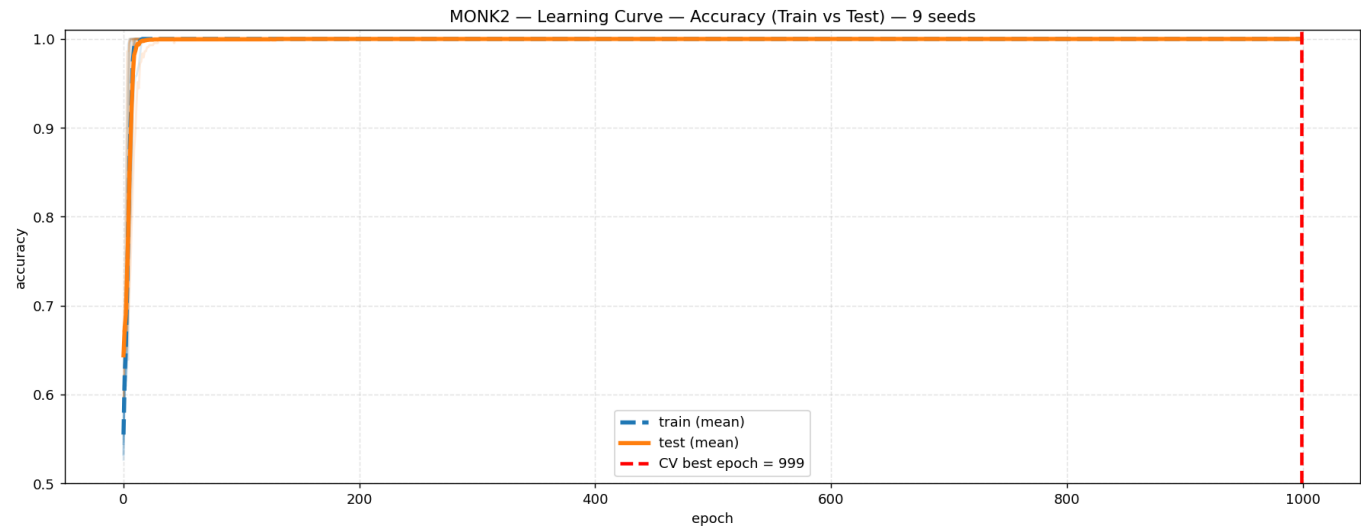
- **MONK1:**
 - **Coarse GS:** 31104 models ~ 45mins
 - **Refined GS:** 1728 models x 5 Folds ~ 20mins
 - **Test:** 1 model x 10 seeds ~ 45sec
- **MONK2:**
 - **Coarse GS:** 31104 models ~ 61mins
 - **Refined GS:** 2304 models x 5 Folds ~ 36mins
 - **Test:** 1 model x 10 seeds ~ 55sec
- **MONK3:**
 - **Coarse GS:** 31104 models ~ 54mins
 - **Refined GS:** 9216 models x 5 Folds ~ 49mins
 - **Test:** 1 model x 10 seeds ~ 55sec
- **CUP:**
 - **Coarse GS:** 3888 models ~ 38mins
 - **Refined GS:** 6750 models x 5 Folds ~ 70mins
 - **Test:** 1 model x 10 seeds ~ 2mins

TASK	HYPERPARAMS.	MSE (TR/TS)	ACC. (TR/TS) [%]
MONK 1	H. Layer Units: 8 Opt. = adam $\eta = 0.2$ mb = 16 Activation: Sigmoid	0/0	100/100
MONK 2	Units: 12 Opt. = adam $\eta = 0.04$ mb = 16 Activation: ReLU	0/0	100/100
MONK 3	Units: [12, 8, 4] Opt. = adam $\eta = 0.04$ mb = 32 Activation: ReLU	0.0007/0.05	99.90/93.95
MONK 3 (reg.)	Units: [12, 8, 4] Opt. = adam $\eta = 0.04$ mb = 32 $\lambda = 0.001$ Activation: ReLU	0.05/0.01	95.08/98.61

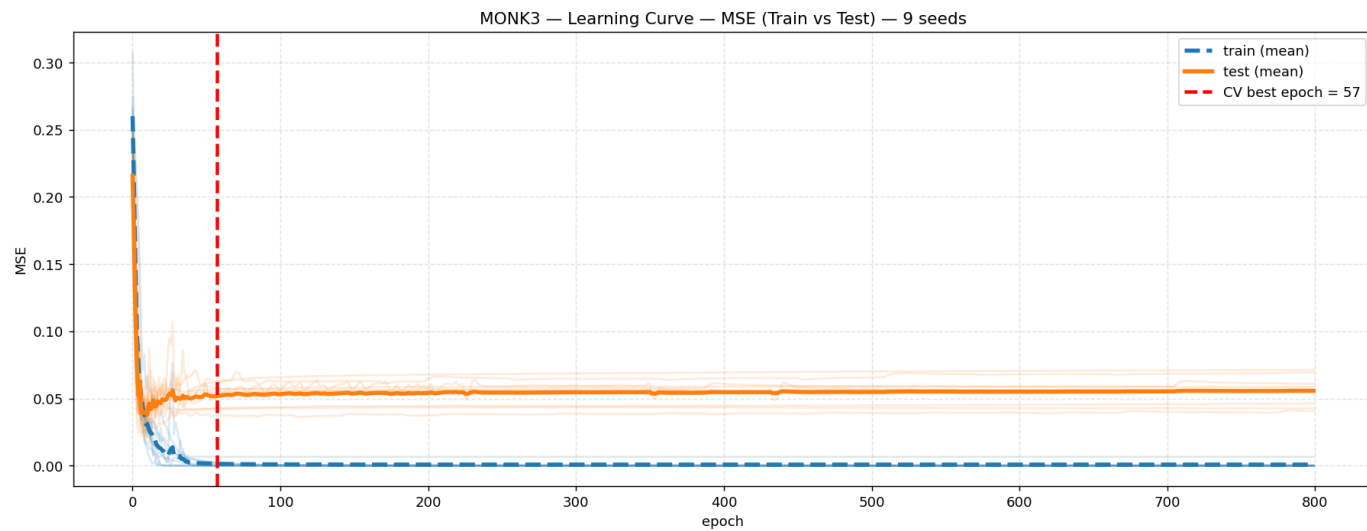
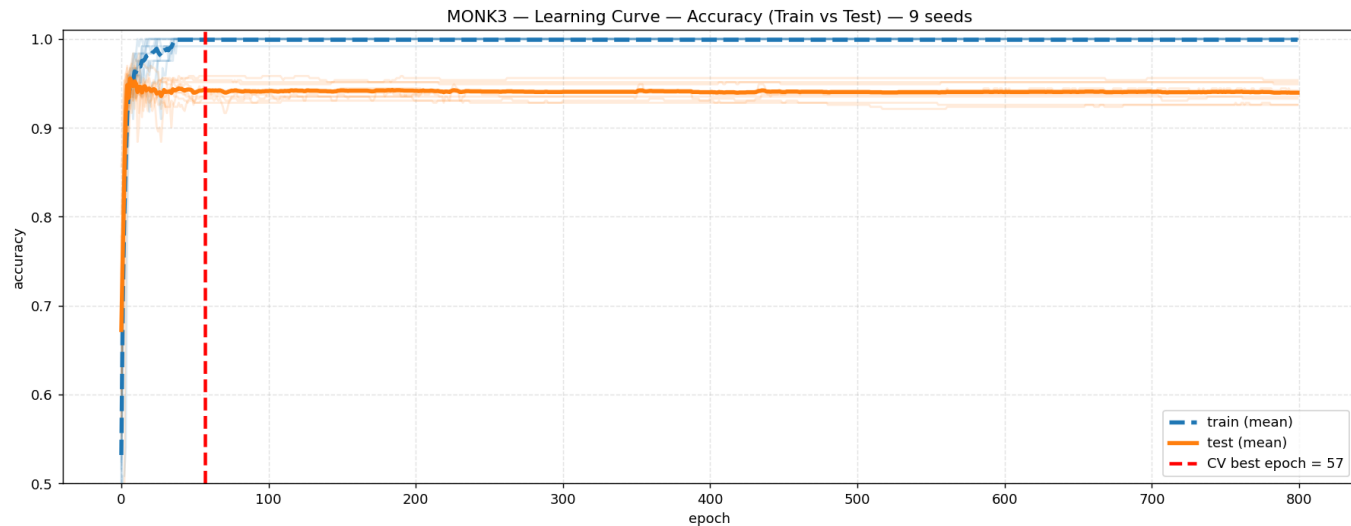
Monk1 Results



Monk2 Results



Monk3 Results



CUP Validation schema: data splitting

Training + Validation (90%)

Internal Test (10%)

- **Model selection on the development set**, 90% of the ML-CUP25-TR dataset
- **Retraining** on the development set
- **Assessment on the internal test set**, 10% of the ML-CUP25-TR dataset (used only once)
- **Final retraining** on the whole ML-CUP25-TR dataset for the blind test set

CUP Validation schema: Model Selection

- **Data Preprocessing:** Input features standardized using **Z-score** (mean=0, std=1)
- **Multi-step grid search:** coarse to fine search for the hyperparameters with the same approach described in the previous slides
- **3888 combinations** tested in the **coarse GridSearch** and **6750** for the **fine GridSearch**
- Synthesis and range of the tested **hyperparameters**:
 - **Hidden units:** 1/2/3 hidden layers
 - **Activation functions:** tanh, ReLU, Sigmoid
 - **Dropout:** [0 .. 0.5]
 - **L2 regularization:** [10^{-5} .. 10^{-2}]
 - **Optimizer:** SGD
 - **Momentum:** [0.5 .. 0.9]
 - **Learning Rate:** [10^{-3} .. 10^{-1}]
 - **Batch Size:** [1 .. 360]

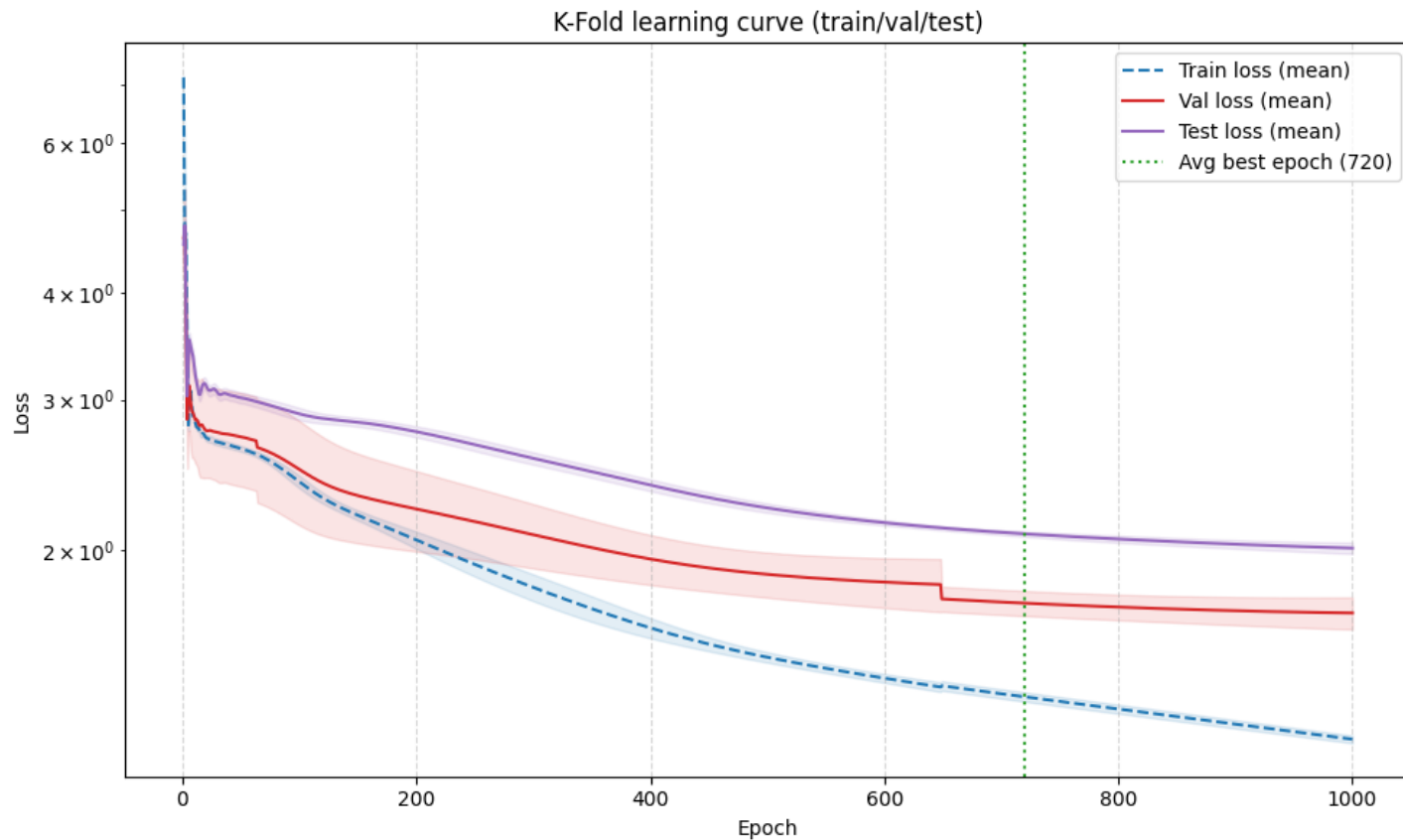
CUP Results - Relevant Choices

- **Z-score Standardization**: applied to all inputs to prevent floating-point overflows and ensure numerical stability during gradient descent.
- **Optimization Strategy**: used SGD with Momentum to accelerate convergence and L2 Regularization to control model complexity and prevent overfitting.
- **Architecture Search**: evaluated both single and double hidden layer configurations to find the optimal capacity for the 4-target regression task.
- **Model Selection Criteria**: final model selected based on learning curve smoothness and stability rather than just on the absolute minimum validation loss, ensuring better generalization.
- **Weight Initialization Ensemble**: The final model is an ensemble that averages predictions from multiple networks initialized with different random seeds. This approach reduces sensitivity to initialization and improves the robustness of the final results.

CUP Results - Final Model Selection

HYPERPARAMS.	SET	MEE (mean $\pm$ std)
Hidden units: [64, 32, 16] Activation functions: tanh Dropout: 0 L2 regularization: 0.0001 Optimizer: SGD Momentum: 0.8 Learning Rate: 0.01 Batch_size: Full Batch (360)	Training	19.92 $\pm$ 3.04
	Validation	22.64 $\pm$ 2.39
	Internal Test	21.64

CUP Learning Curves

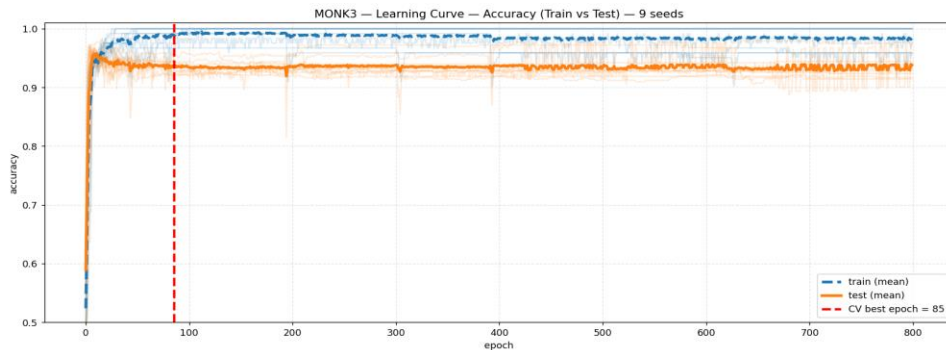
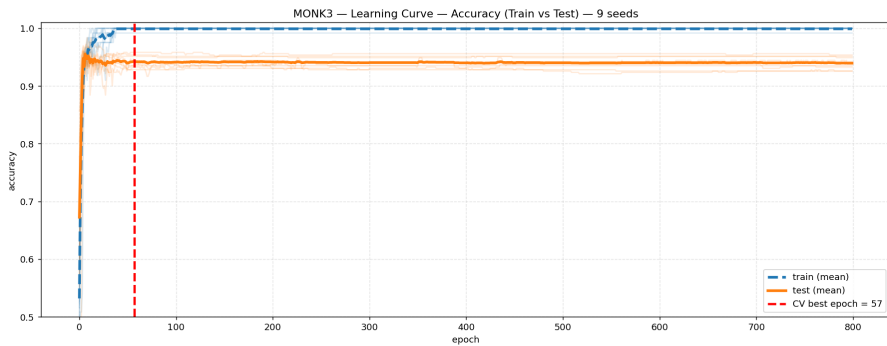


Final Discussion - Key Findings and Strong Points

Two-phase model selection (coarse → refined): reduced search space while enabling statistically performing region selection

Multi-seed ReTraining + Ensemble majority voting: more reliable predictions than any single model, reduced sensitivity to initialization and reduced variance across runs, improved stability

EXAMPLE: MONK 3 Test – Top 1st Model vs Top 3rd Model



"model": {"activation": "relu", "dropout": 0.0, "hidden_units": [12,8,4]}, "optim": {"lr": 0.04, "momentum": 0.0, "name": "adam", "regularizer": {"l2": 3e-07}}

{"activation": "relu", "dropout": 0.0, "hidden_units": [4, 4]}, "optim": {"lr": 0.04, "momentum": 0.3, "name": "adam", "regularizer": {"l2": 1e-06}}

Conclusions – What we have Learnt

- **Mathematical Understanding:** Full manual implementation of **forward and back propagation**, handling of gradients, loss functions, and optimization dynamics
- **Software & Architecture Design:** Modular, reusable, and reproducible **neural network framework**
- **Experimental Methodology (★ Golden Rule):** Strict separation of **training / validation / test** data, **Two-phase model selection (coarse → refined)** and Model Assessment on Test set for final unbiased evaluation

Blind Test Results: iCavalli_ML-CUP25-TS.csv - TEAM: iCavalli

Bibliography

1. [Scikit-learn: Machine Learning in Python](#), Pedregosa *et al.*, JMLR 12, pp. 2825-2830, 2011.
2. N. Morgan, H. Bourlard, *Generalization and Parameter Estimation in Feedforward Nets: Some Experiments*, in Advances in Neural Information Processing Systems 2 (ed. D. S. Touretzky), Morgan-Kaufmann, pp. 630–637, 1990.
3. Varma, S., & Simon, R. (2006). *Bias in error estimation when using cross-validation for model selection*. BMC Bioinformatics, 7:91.
4. Nguyen, D., & Widrow, B. (1990). *Improving the learning speed of 2-layer neural networks by choosing initial values of the adaptive weights*. In Proceedings of the 1990 IJCNN (International Joint Conference on Neural Networks).
5. Harris, C.R., Millman, K.J., van der Walt, S.J. et al. *Array programming with NumPy*. Nature 585, 357–362 (2020). DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2). ([Publisher link](#)).
6. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research (JMLR)*, 15, 1929–1958.